本章路線

這章要回答的核心問題是：在沒有共享記憶體、時鐘不一致且節點可能失效時，系統如何提供可理解且可恢復的服務？

前置知識。 network stack、RPC、process 與 file-system interface。

- 比較網路型 OS 與 distributed OS
- 分析 naming, routing, RPC 與 failure semantics
- 檢查 DFS cache consistency 與 availability

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Message-passing 系統的目標與限制

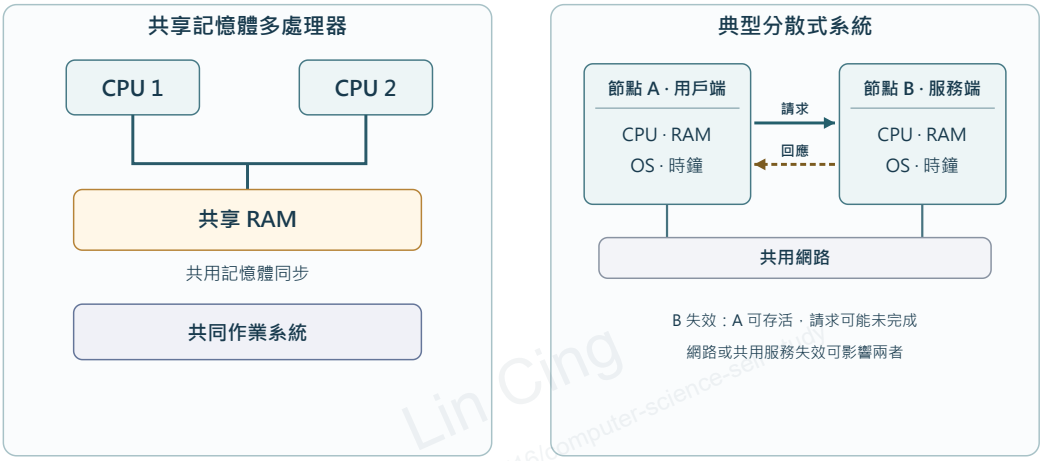
本章以典型 message-passing 模型為例：每個 node 有本機 memory 與 OS，透過訊息通信；不假設共享實體記憶體或完美共同時鐘。local 與 remote 是相對於請求者而言。分散式共享記憶體可作為另一層抽象，時鐘也可同步，但同步仍有誤差。資源分享、計算加速、可靠度與通信是設計動機，不是分散後自動得到的性質。

固定速度模型：單機 100 s 工作中 20 s 串行、80 s 可平均分給 4 nodes，另加 5 s 協調；$T_4 = 20 + 80/4 + 5 = 45$ s，加速比 $100/45 \approx 2.222$。忽略協調的 Amdahl 上限為 $100/20 = 5$；加入 node 不能消除串行部分，網路或協調成本增加還可能使系統更慢。

在**固定的一次讀取時刻**，三個可互代副本各自不可用的事件獨立且機率 $p = .01$；假設網路及 metadata 服務可用，任一可用副本皆能讀取有效資料，則讀取不可用的機率為 $p^3 = 10^{-6}$。若共用電源、rack 或依賴同一 metadata leader，此獨立模型可能不成立。期間內三個副本曾各自失效，並不代表它們同時不可用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享記憶體與訊息傳遞



兩者是模型：效能與可靠度取決於瓶頸、負載及故障域。

共享記憶體模型以同步機制協調 CPU；分散式模型中各節點透過網路交換請求與回應。服務端失效或網路中斷時，用戶端可能無法判斷請求是否已執行。共享網路與服務也可能成為共同故障點；分散式共享記憶體是另一種可建立的抽象。

Network OS 與 distributed OS 的透明範圍

network OS允許明確連遠端host或copy檔案，使用者知道機器邊界；distributed OS/中介軟體可提供更一致的命名、存取與遷移介面。透明性不是把遠端latency/failure從物理上消除，而是把定位/調用方式包進介面並定義失敗語義。

remote login在遠端執行session，file transfer建立另一份copy，DFS則提供遠端共享檔案語義；三者不能互稱。SSH/SFTP可保護對應session，仍需驗host身分與user授權；一般FTP本身的傳輸/認證不應被當作加密通道。

匿名file service需獨立限制namespace與操作，不能由名為anonymous就假定隔離。存取命名、帳號映射、網路失敗與多份copy的更新衝突都必須有規則。

Data migration 與 computation migration 的實算

資料1 GiB、結果1 MiB、有效吞吐100 MiB/s、單次RTT20 ms。把全資料搬來至少1024/100+.020=10.260s，再做local compute；把計算送去，若remote compute=.500s且request可忽略，回結果需.010s，含一次RTT共.530s。比較須含remote CPU queue、啟動/序列化與結果大小，不能只說「大資料一定搬計算」。

whole-file caching將首次成本前置，後續local access便宜；on-demand block caching只搬所需區塊。若只read 1 MiB，整個1 GiB的資料搬移多1023 MiB；若讀全檔且逐block各一RTT、又沒有pipeline，很多小RPC可能更慢。

process migration要轉移或重建PC/register、memory、open files/socket等外部references與執行環境；只把程式碼傳到另一機啟動，屬remote execution/computation placement，不等於帶完整執行狀態的process migration。跨ISA/ABI更需要相容層，不能直接搬register值。

LAN／WAN：範圍不是單一速度規格

LAN常在較小管理/地理範圍，WAN跨站點；真實capacity/latency取決於link、distance、queue與負載。switch、router、wireless AP有不同職責；現代switched Ethernet不應一概描述成所有host共用一條collision bus。LAN可能較擁塞，WAN backbone也可能有高bandwidth，不能只按名稱排序吞吐。

latency≈serialization L/R + propagation + queue + processing。教學例1500B packet、1 Gbit/s，serialization=1500×8/10^9=12 μs；單向propagation5ms遠大於serialization。把link升10倍只把12μs降1.2μs，不能把5ms propagation也除10。

bytes、bits與rate單位分開：100 Mbit/s=12.5 MB/s（十進位，尚未扣headers）；MiB/s使用2^{20} B分母。wireless名目rate也不等於application goodput；本章數字皆為指定模型，不列產品當前最高規格。

Bandwidth-delay product 與在途資料

link100 Mbit/s、RTT40ms，BDP=100×10⁶×.040=4×10⁶ bits=500000B≈488.28KiB。若可靠stream最多容許64KiB未確認資料，理想window bound=65536×8/.040=13.1072 Mbit/s，即使link有100Mbit/s也可能填不滿。

這是無loss、固定RTT、無其他瓶頸的上界模型；TCP的congestion window、receiver window、ACK行為、slow start與retransmission還會影響實際結果。增加send buffer只可能解除一種限制，不保證消除congestion或server CPU瓶頸。

低latency RPC與大檔stream的效能需求不同：一個只回16B的RPC可能主要花RTT，不是16B的serialization。這解釋為何資料/計算擺放與cache對分散式OS至關重要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Naming：domain name、IP、port、object id 分開

`store.lab.test` 是 domain name，不是 IP address；解析可得 IPv4 如 198.51.100.20 或 IPv6。IP 位址不直接等於 user 或永久 machine identity；service 還需 transport protocol、port 與應用層名稱。TCP connection 通常由來源／目的 IP 與 port 及 protocol 區分，port 不是 process PID。

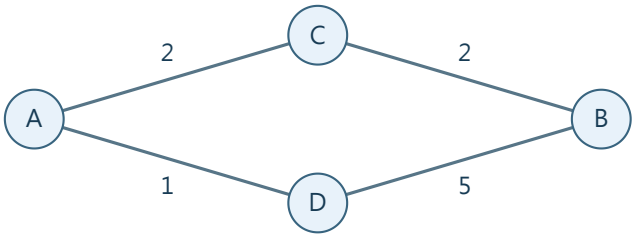
DNS 使用 resolver、query、cache 與 delegation，不要求每個 label 都有獨立 name server。假設 root 委派 test、test 委派 lab.test，resolver 在 cache miss 時由已知 root hints 沿 referrals 查詢權威 A record。以下限定遵守 TTL 的教學 resolver，沒有 serve-stale 或本機固定 override；此限定不是所有實作的通則。

若 t=0 取得 A=198.51.100.20、TTL=600 s，t=100 剩 500 s；權威在 t=200 改為 .30 時，舊 cache 仍可於 t=600 前使用 .20。到期後若重新 query 成功且記錄未再改動，才取得 .30；查詢失敗不能稱其為已驗證的新值。DNS 資料完整性與連線對端認證另需相應機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Routing：用同一加權圖追下一跳

同一張非負加權圖：distance 更新不跳步



從 A：D=1 · C=2 · B=∞

確定 D：B=1+5=6

確定 C：B=min(6,2+2)=4

確定 B：A → C → B · cost4

若 C–B 失效 · 重算走 A→D→B · cost6；動態協定仍有偵測與收斂時間。

原創graph：A–C=2、C–B=2、A–D=1、D–B=5。Dijkstra從A：初始dD=1、dC=2、dB=∞；取D後dB=6；取C後改dB=4；取B確定。最佳A→C→B，cost4。若C–B失效，重新算後走A→D→B，cost6。

static routing按預設規則/管理更新；virtual circuit固定某session路徑；dynamic routing由協定更新可達性/metric。dynamic不等於每packet一定挑當下最空link，也不保證收斂期間無loop/drop。沿同一路徑僅在link/queue等保持相應次序時保序；end-to-end stream ordering仍由transport協定實現。

control plane計算路由，data plane按forwarding table處理packet；SDN可分開這兩者，仍需更新一致性、失敗處置與保護管理通道。

Packet 與 switching：有狀態連線不等於專用電路

packet/frame/datagram可有可變長度，各層各有header與大小限制；一個application message可拆多segment/packet，也可多message共一TCP read。TCP connection不保留專用physical link，UDP也可由應用另建重傳/可靠訊息協定。

circuit switching預留相應路徑資源；message switching整個message store-and-forward；packet switching可把message分段pipeline與統計多工。兩串行links各1Mbit/s、忽略propagation/header/queue：4000B整message走兩hop需2×4000×8/10⁶=64ms；拆4個1000B packet，每packet每hop8ms，pipeline完成需(2+4−1)×8=40ms。

若每packet另加40B header，最後一個packet到達需5×1040×8/10⁶=41.6ms；整message若只加一次40B則需64.64ms。這是固定store-and-forward模型，不能用其公式直接推所有cut-through/circuit網路。

OSI 是職責模型，不是每層一定一顆硬體

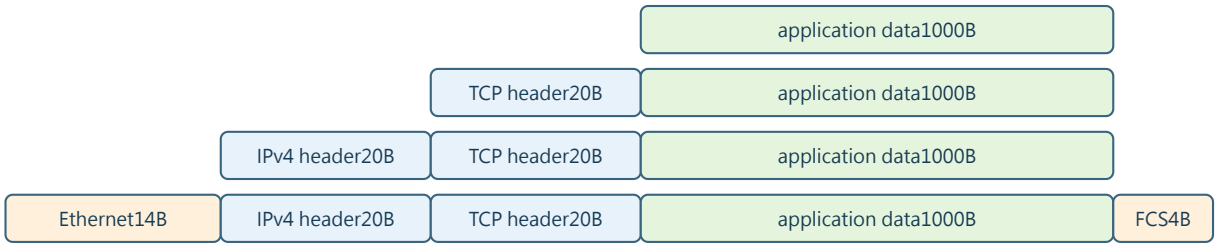
physical傳signal/bits；link在相應link傳frames；network做logical addressing/forwarding；transport提供process endpoints間服務；session管理dialog；presentation處理表示/encoding；application定義服務操作。真實TCP/IP常把session/presentation功能置於application libraries/protocols，不能據層數少就定理式推出更有效率。

TCP提供ordered reliable byte stream與flow/congestion control，不保留application message邊界；UDP保留datagram邊界但不內建同樣的交付/排序保證；IP是best-effort datagram。NIC offload可把部分checksum/segment工作移到硬體，所以「下三層一定全硬體、上四層一定全軟體」不是通用規格。

序列化需定欄位長度、byte order、encoding與版本；不能把C struct含padding/pointer原樣傳到異構機器就當跨平台協定。

Encapsulation：同一份 payload 的長度守恆

每層加入自己的 header / trailer；長度逐層核對



欄寬非比例；IP total1040B · MAC frame1058B；preamble/SFD與IFG另計。

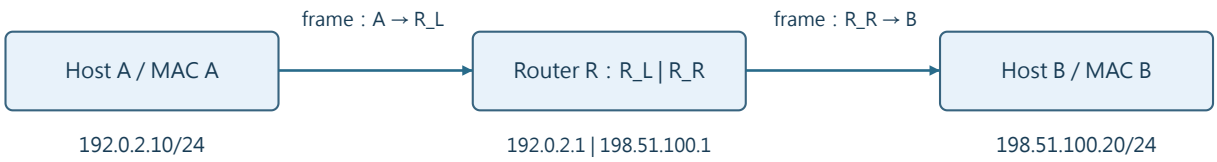
固定IPv4無options20B、TCP無options20B、application data1000B：TCP segment1020B，IP total length1040B，EthernetII header14B+IP1040+FCS4=1058B。未含preamble/SFD8B與指定模型的IFG12 byte-times，若連續傳送的wire占用為1078 byte-times。

IPv4 MTU1500時，本例1040可放一frame；MSS通常從可用IP payload扣TCP header，不能把1500全當application bytes。加VLAN/options/tunnel/security headers後重新核算，不延用上述數字。

TLS或其他認證/加密有明確endpoints；router一般不因轉送就成為application session的認證對端。lower-layer checksum主要偵測傳輸錯誤，不是對惡意變更的secret-key authenticator。

IPv4 跨兩個 Ethernet LAN 的完整路徑

無 NAT 的跨 LAN 封裝：每 hop 的 MAC 不同



A 對 next-hop 192.0.2.1 做 ARP，不在左 LAN 對遠端 B 做 ARP。

IP：192.0.2.10 → 198.51.100.20 保持；router TTL64→63，IPv4 checksum 與 link FCS重算。

固定無NAT/tunnel：A=192.0.2.10/24、MAC A；router R左介面192.0.2.1/MAC R_L，右介面198.51.100.1/MAC R_R；B=198.51.100.20/24、MAC B。A查DNS取得B地址，/24比較發現非local，route選next-hop192.0.2.1。

A沒有R_L的ARP entry時，向本LAN broadcast「誰有192.0.2.1」，R回答R_L；A送Ethernet dst=R_L/src=A，內含IP dst=B/src=A。不能在A的LAN直接ARP遠端B。router查route、TTL64→63並更新IPv4 header checksum，查/ARP右LAN B的MAC，再封裝dst=B/src=R_R，IP endpoints仍A→B。

B解封裝後交IP，再按TCP/UDP port/connection送給對應socket。ARP broadcast通常不由router轉送到別的L2廣播域；ARP回答也不是密碼學身分證明。

哪些 header 改，哪些通常不改

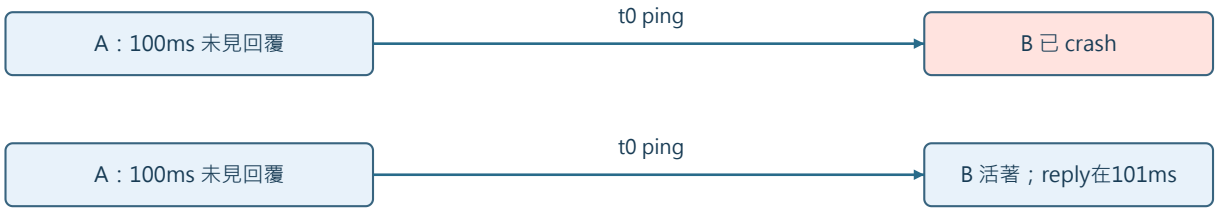
上例每hop Ethernet src/dst與FCS重建；IPv4 TTL/checksum變，無NAT/tunnel且無其他轉換時src/dst IP保持；TCP端點與payload保持。故不能說跨router時「只有Ethernet header變，其他headers全部不變」。若TTL到0，router不能繼續正常forward；錯誤回報也可能被網路遺失/過濾。

普通無tag EthernetII採6B destination MAC、6B source MAC、2B EtherType、46–1500B data/pad、4B FCS，從destination到FCS最小64B。data不足46需pad，但IP total length不把pad當IP資料。preamble/SFD在此frame長度分母外。

原創短TCP payload4B：IPv4 20+TCP20+data4=44B，需2B link pad，frame=14+46+4=64B；加8B preamble/SFD與本例96-bit IFG，共84 byte-times，在100Mbit/s占6.72µs。IFG是空閒時間不是附進IP packet的12B資料；本例不泛化到所有高速PHY的gap容差。

Failure detector：timeout 只能先產生懷疑

相同本地觀察，兩種不同遠端狀態



100ms觀察窗內，A 的 event history相同。沒有delay上界，timeout不是crash證明。

在沒有已知訊息／排程延遲上界的 asynchronous 模型，A 於 t=0 發 ping，t=100 ms 仍無 reply，不能分辨 B crash、link partition、message loss 或 B 只是很慢。兩條執行可讓 A 的本地觀察完全相同：① B 已 crash；② B 仍活但 reply 延到 101 ms。任何只依這段觀察便斷言 crash 的算法，都會在②誤判。

另考慮有界教學模型：B 持續運作，heartbeat 的**名義排程時間**每 H=1 s 一次，每次實際發送比其名義時間遲 0–s，s=.2 s；每筆訊息都送達，單向延遲在 0–δ，δ=.1 s。接收處理與計時不增加延遲或漂移。前一筆可在名義時刻立即到達，後一筆最晚在下一名義時刻再加 s+δ 到達，故到達間隔至多 H+s+δ=1.3 s；從上一次到達開始的 1.5 s timeout 在此模型不會誤報活節點。若發生丟包、partition、延遲超界或計時假設不成立，便不能套用此保證。

Reconfiguration：選新 leader 還要擋舊 leader

A 原持 shared storage 寫權；partition 後 B 因 timeout 自稱新 leader，而 A 仍在運作，雙方寫同一份資料會造成 split brain。心跳與本地宣稱「我是 leader」不能保證 mutual exclusion。

教學 fencing 模型中，協調服務正確核發單調 epoch；storage 安裝 currentEpoch=43 後才讓 B 操作。**每筆 write 的 token 驗證與其提交，必須和 epoch 安裝在同一線性化順序中原子執行，或由等價序列化機制保證。** 若 A 的 epoch 42 write 在安裝 43 前已線性化提交，它屬於舊時期；若排在安裝 43 之後，儲存端必須拒絕。只在提交前某個時點檢查一次並不足夠：舊寫入可能先通過檢查，延後到安裝新 epoch 後才落盤。

此保證還要求每條寫入路徑執行驗證、核發與安裝正確，不能讓不同 partition 各自遞增數字就宣稱取得合法權限。經確認的硬體或存取隔離也可擋舊節點。恢復 A 時先同步已提交狀態、成員／epoch 與未完成操作，再作 follower；request id 或 generation 可防止重啟後錯配舊回應。

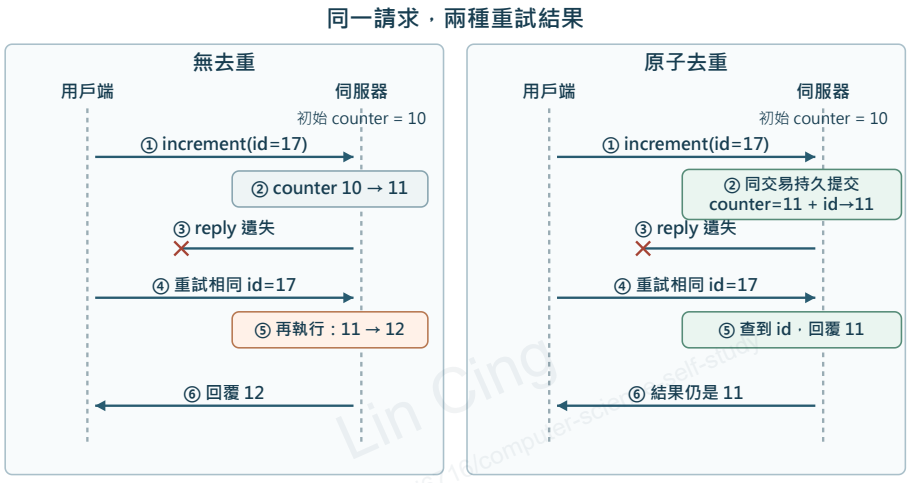
RPC retry：遺失 reply 不等於沒有執行

原創counter=10，client呼叫increment(id17)。server改成11後reply丟失；client timeout重試若直接再執行變12。TCP重傳提供同connection的byte交付，不替應用跨重連去重，不能由用了TCP推exactly-once RPC。

明確的教學at-most-once模型：server交易中查durable table[id]，若存在回保存result；否則把counter++與table[id]=result**原子持久提交**再reply。重試id17得到11而不再加。若先改counter後crash、尚未記id，重試仍可能加第二次；若先記id後未改counter，又會遺失效果。

此模型保證在table與業務狀態同交易/恢復域、id唯一且未過早忘記的前提下最多提交一次。client在network永久中斷時仍可能不知道結果；發電郵/外部付款不在同交易內時，需要額外協定，不能直接沿用本例結論。

RPC 重試：回覆遺失與去重



前提：ID 在重試範圍內唯一且記錄持續保留；外部效果不屬此交易，須另設協定。

同一事務持久提交計數更新與 `id→result`；重試沿用唯一 ID，伺服器返回已存結果。去重表必須保存至所有可能重試結束；郵件或付款等交易外效果需要額外協定。

Fault tolerance 的容量與故障域

兩台active-active servers各可100 requests/s，平時合計負載160；失一台後容量剩100，至少60/s不能穩態消化。只有replica可接手不代表failover仍達原SLO，必須預留容量、load shedding或降級。

stateless worker容易重啟，但輸出副作用仍可能重複；stateful service還要log/recovery/replication/鎖。RAID處理部分disk failures，不等於跨站點服務可用或logical corruption有backup；cluster/replica需要配合獨立故障域與一致性機制。

recovery先重建durable committed state，辨識/回復in-flight operations，再重新接流量。不能把「還沒收到commit reply」一律當成「未commit」，重試策略須依查詢/去重契約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transparency 與 scalability 要寫可觀察規格

location透明隱藏在哪裡，migration透明讓搬動後名稱/操作仍可用，replication透明隱藏多副本；failure透明不能無限制隱藏永遠不回覆的remote operation。user mobility還需身分、home/environment與應用session一致，不只是同名帳號存在。

原創資源上限：每worker100requests/s，10workers共1000；metadata coordinator每request花2ms CPU、只有一core，上限500/s。第11個worker不提高這個500/s界；cache/批次/partition metadata若改每request資源量才可能解除瓶頸。增加replicas的同步/重建traffic也要算入容量。

HDFS歷史案例固定Hadoop1.2.1：NameNode管namespace與block locations，DataNodes保存/傳輸blocks，client資料路徑不經NameNode。大block/streaming/較簡化的寫入模型適配大量資料，不能把它當作完整POSIX共享檔案系統或任意低延遲交易資料庫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

HDFS 1.2.1 的明確副本與資料流模型

本例自訂block64 MiB、replication factor3；檔案130 MiB分64、64、2共3blocks，logical130、payload副本總390 MiB（不含metadata、checksum、padding/配置額外）。最後2 MiB不是強制占滿64 MiB資料流；metadata仍須為該block定位。

client向NameNode取得block與DataNode位置，再直接向DataNode讀；寫入可沿DataNode pipeline複製。DataNode heartbeat/block report讓metadata知道狀態，缺副本時安排re-replication；失效偵測仍有延遲，重建也消耗network/disk。

本版本的Secondary NameNode主要協助checkpoint，不等於自動接替active NameNode的hot standby。副本分散到故障域能改善data availability，但NameNode/metadata與client可達性仍要納入，不由factor3推無條件高可用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS naming：路徑、穩定身份、位置分三層

location transparency是名稱不暴露物理位置；location independence是搬位置後名稱仍不需改。`serverA:/x`把server編進名稱；mount遠端目錄可讓client路徑較透明，但各client可掛到不同位置；global namespace則再統一命名/管理規則。

原創兩級映射：`/course/a.txt`→FID(volume7,file12,generation3)；volume7→serverA。搬volume到serverB只改第二層，不改path/FID。file12刪後再配置需換generation，否則舊FID可能錯指新檔。快取path→FID與volume→location有不同失效條件，不可一起永久保留。

replicated/cached location map更新仍需一致性/版本/失效規則；多一層indirection沒有消滅更新問題。diskless boot在DFS client尚未載入前，需firmware/初始boot protocol先取得kernel，再啟DFS；不能用尚不存在的DFS程式去載入自己。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Remote service 與 cache 的完整成本例

100次4KiB reads；無cache每次remote cost5ms，共500ms。現在10miss各5ms、90hit各.1ms，共50+9=59ms；payload network bytes10×4KiB=40KiB，相對無cache400KiB。這個例已指定hit/miss與remote cost，未把實測工作負載命中率當作定理。

若每次還要1ms server validation，總另加100ms→159ms；「有cache」不自動避開network round trip。cache granularity大可能read-ahead改善後續hit，也可能單byte讀造成大量多餘transfer；cache總容量固定時，大單位還會減少可存blocks。

client memory cache快但斷電易失；disk cache能保存已durable內容，仍需恢復metadata與驗證新鮮性。server自己也有cache；client寫到server RAM不等於已到stable storage。write-through/write-back描述何時向下一層推送，需另定那一層返回的持久性。

Cache consistency：以時間線說出保證

server新版本與client cache可見性是不同時間線



只定期驗證會有stale窗口；強callback/lease模型須在write完成前收回衝突讀權。

client-initiated：C1在t0讀v0並驗valid；C2於t3完成v1；C1若只在t10再驗，t4仍可能讀cache v0。這不是「每read都看最新提交」。在open才驗，則必須界定同一open期間的可見性；在每access驗則增加network成本，且validation與實際read仍需協調避免競爭窗口。

server-initiated：server追蹤誰持cache/何種讀寫權，以callback失效/回收許可，或衝突時停cache轉remote service。若要強一致教學模型，write完成前須確認所有衝突read權已收回/失效，或其受控lease已到期；partition時不能一邊讓舊client無限讀舊值、一邊宣稱新write已對所有人立即可見。

書中AFS/OpenAFS的callback避免每次都驗server，但應用已讀進自己的buffer不因callback自動改值；save/close與cache manager取新資料的時點影響使用者觀察。不同DFS的session/close-to-open/更強語義不能互當同義詞。

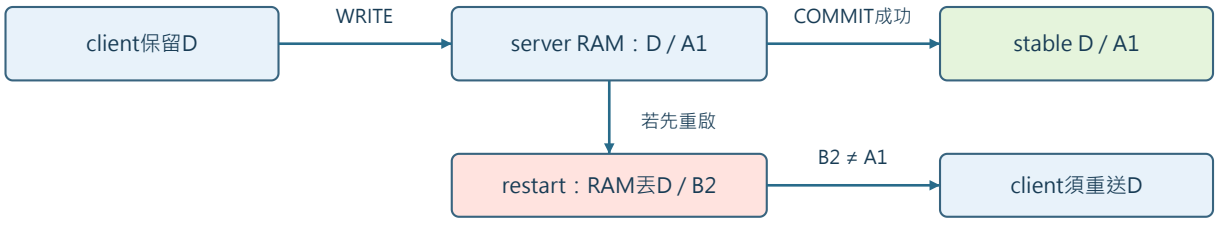
NFSv3：stateless 不表示沒有 cache，也不表示每次 WRITE 已落盤

固定RFC1813 NFSv3：READ/WRITE攜filehandle、offset、count等，核心檔案I/O不依賴server為每次open保存同等session state；server仍可有cache、duplicate-request state，其他鎖/協定另有責任。client permission/authentication與filehandle生命期仍要處理。

WRITE要求/回報有UNSTABLE、DATA_SYNC、FILE_SYNC等穩定程度；client不能把UNSTABLE成功當作所有資料已durable。延遲寫入若需後續COMMIT與verifier比對，必須保留可重送資料。weak cache consistency attributes有助判定變更，不單獨保證所有clients嚴格線性一致。

NFSv3 unstable-write 的重啟 trace

NFSv3：相同WRITE成功之後，正常與重啟有不同結果



UNSTABLE 後保留資料；成功 COMMIT 須比對 verifier。
重傳後可由符合所需穩定程度的 WRITE 確認實際寫入範圍。

原創4KiB資料D：t0 client送WRITE(offset0,D,UNSTABLE)，server RAM接收，回committed=UNSTABLE、verifier=A1；client保留D。正常路徑t1 COMMIT成功且verifier仍A1，server已按契約保存到stable storage，client才可清這批重送狀態。

故障分支：t0之後server重啟丟RAM未提交D，新的service instance用B2。client後續COMMIT/WRITE看到B2≠A1，不能假稱原UNSTABLE資料已保留；需重送尚未保證stable的D，再取得相應穩定結果。不是簡單地「COMMIT回success就不必看verifier」。

這個協定保障依賴client仍有重送資料、server遵守stable/verifier契約及正確處理範圍；不證明不受限concurrent writers會有應用想要的交易語義。durability、cache visibility與RPC去重是三個不同問題。

把一個遠端操作接回所有層

service name→DNS address→route next hop→ARP MAC→link frame/IP packet/transport endpoint→RPC id→服務授權與durable commit→client cache。每層的成功只支持該層契約：能送frame不代表RPC完成，收到TCP bytes不代表業務只執行一次，NFS UNSTABLE不代表落盤。

分散式系統的「透明」是在明列邊界內提供一致介面；latency、partial failure、資料副本與外部副作用不能用介面名稱消除。正確設計要在觀察不足時保留unknown/suspected，而不是把timeout當作邏輯證明。

速度用serial/parallel/coordination拆分；網路用bytes/bits、headers、RTT與window對分母；可靠度先畫故障域；RPC重試以request id與原子記錄推導；DFS把name/object identity/location、visibility/durability分開。

一般保證取決於允許的state transitions：heartbeat在無界delay不能準確斷crash，fencing依賴資源端強制epoch，at-most-once交易依賴去重紀錄不被過早忘記。原創練習把反例、正常路徑與重啟後中間狀態都列出，讓讀者可逐步驗算。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 17. RFC 1034, RFC 826, RFC 1813. Hadoop 1.2.1 HDFS. OpenAFS.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

把一個遠端操作接回所有層 （續）

https://github.com/xinhi6716/computer-science-self-study